GeekBrains

Архитектура компьютера и основы ОС Linux

Логические диски

MBR. Работа с MBR. Утилиты. GPT.

Оглавление

Разделы, форматирование

- MBR
  - Структура MBR
  - Таблицы раздела (4 primary)
- GPT
- Зачем нужна таблица разделов
- Форматирование
  - Низкоуровневое форматирование
  - Высокоуровневое форматирование
  - Восстановление информации
  - Удаление без возможности восстановления
- Утилиты для работы с дисками
  - Утилита fdisk
  - Утилита dd
- Утилита hexdump
- Работа с утилитами fdisk, dd

Практическое задание

Дополнительные материалы

Используемая литература

Разделы, форматирование

Для установки нескольких операционных систем на один жесткий диск или для разделения хранимой информации (системной, файлов, архивов) применяется разметка диска на разделы. Каждый раздел

© geekbrains.ru

выглядит для системы как отдельный диск и имеет в DOS и Windows отдельную букву диска (например, диски C: и D:).

Для управления разделами используется таблица разделов (partition table) — служебная область в начале жесткого диска, которая описывает дисковую разметку. Каждый раздел может содержать свою файловую систему и использоваться для загрузки размещенной в ней операционной системы, если в этом есть потребность.

MBR

MBR (Master Boot Record — главная загрузочная запись) содержит исполняемый код, необходимый для передачи управления загрузчик и таблицу разделов (partition table). Занимает 512 байт, только умещается в 1 сектор. MBR может содержать только 4 первичных (primary) раздела, а при необходимости увеличения их количества вместо одного из первичных разделов можно создавать расширенный раздел (extended partition). Такой раздел будет содержать внутри себя еще несколько (до 16) логических разделов (logical). С созданием разделов и установленных на нем ОС связаны ограничения: например, Windows необходимо обязательно устанавливать в первичный раздел, помеченный как активный (загрузочный). Для разделов Linux таких ограничений сейчас нет.

При запуске системы BIOS читает 1 сектор и начинает выполнять код из MBR, программа которого передает управление загрузчику операционной системы. В MS DOS загрузчика нет, и управление сразу передается **io.sys**, который находится в особом месте, до оглавления файловой системы. В Linux передает **Lilo** (в ранних версиях) или **Grub2** (в современных), в Windows — **NTLDR**. Загрузчик Grub2 позволяет держать на одном диске несколько операционных систем (например, Linux и Windows на разных разделах). NTLDR умеет устанавливать несколько версий разных ОС WIndows, но не умеет загружать Linux. При установке Windows GRUB2 будет перезаписан загрузчиком NTLDR. При установке же GRUB2, если обнаружен NTLDR, он переносится в другое место и сохраняется возможность передать ему управление. Поэтому, если нужно на одном компьютере установить и Linux, и Windows, Windows надо устанавливать раньше. Но не забывать, что под разные ОС необходимо разные разделы. В обратной последовательности нельзя, потому что MBR на физический диск один.

Структура MBR:

- 446 байт — код загрузчика;
- 64 байта — таблица основных разделов (Primary);
- 2 байта — сигнатура (подпись). Должна быть 55AAh.

Если сигнатура не равна 55AAh, значит, MBR поврежден.

Ниже приведена наиболее часто используемая структура MBR. Но так как главное в MBR — это не сама таблица разделов, а код загрузчика, для полноценной работы важно, чтобы BIOS мог передать код управления загрузчику. Поэтому есть другие варианты MBR, где код программы может занимать даже все пространство, иметь другую структуру разделов или содержать их в другом месте. Так как таблица разделов в таком случае — не совсем точный критерий содержимого первого сектора, то его называют не MBR, а MBS (Master Boot Sector). В DOS, Windows, Linux MBR используется (либо его заменил более новый GPT).

Структура MBR

Смещение	Длина, байт	Описание
0000h	446	Код загрузчика

01BEh	16	Раздел 1	*Таблица разделов*
01CEh	16	Раздел 2	
01DEh	16	Раздел 3	
01EEh	16	Раздел 4	
01FEh	2	Сигнатура (55h AAh)	

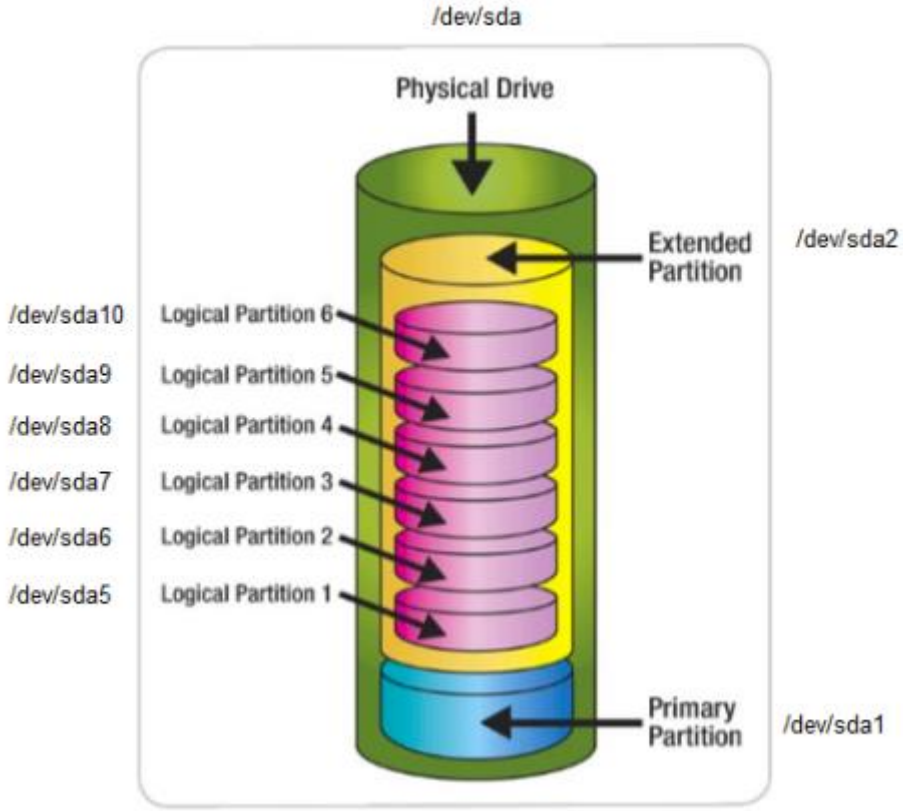
После первых 446 байт кода загрузчика идет таблица разделов 64. 4 блока по 16 байт каждый.

Эта структура данных определяет, какого размера будет каждый логический диск.

Таблицы раздела (4 primary)

Длина (количество байт)	Описание
1	Признак активности раздела
1	Начало раздела — головка
1	Начало раздела — сектор (биты 0–5), цилиндр (биты 6, 7)
1	Начало раздела — цилиндр (старшие биты 8, 9 хранятся в байте номера сектора)
1	Код типа раздела
1	Конец раздела — головка
1	Конец раздела — сектор (биты 0–5), цилиндр (биты 6, 7)
1	Конец раздела — цилиндр (старшие биты 8, 9, хранятся в байте номера сектора)
4	Смещение первого сектора
4	Количество секторов раздела

Максимальное количество основных или первичных разделов (Primary) — 4, потому что в MBR есть всего 4 блока для их описания. Каждый первичный раздел также имеет ограничение по размеру. На количество секторов выделено 4 байта. Каждый сектор может иметь раздел до 512 байт, отсюда максимальный размер раздела — это $2^{32} \times 512 = 2$ TiB.



Пример разметки диска с использованием MBR

Так как MBR позволяет использовать не более четырех первичных разделов, но иногда требуется больше, создаются логические разделы. В этом случае один из первичных разделов помечается как расширенный и в нем содержатся логические системы. Не все ОС умеют создавать логические разделы загрузочными. В WIndows, чтобы система могла загрузиться с раздела, он должен быть физическим.

GPT

На смену MBR пришла таблица GPT — GUID Partition Table. GUID — Globally Unique Identifier — 128-битный статистически уникальный идентификатор. Статистически уникальный, потому что для двух разных GUID вероятность совпадения низкая.

Пример GUID: **{6F9619FF-8B86-D011-B42D-00CF4FC964FF}**

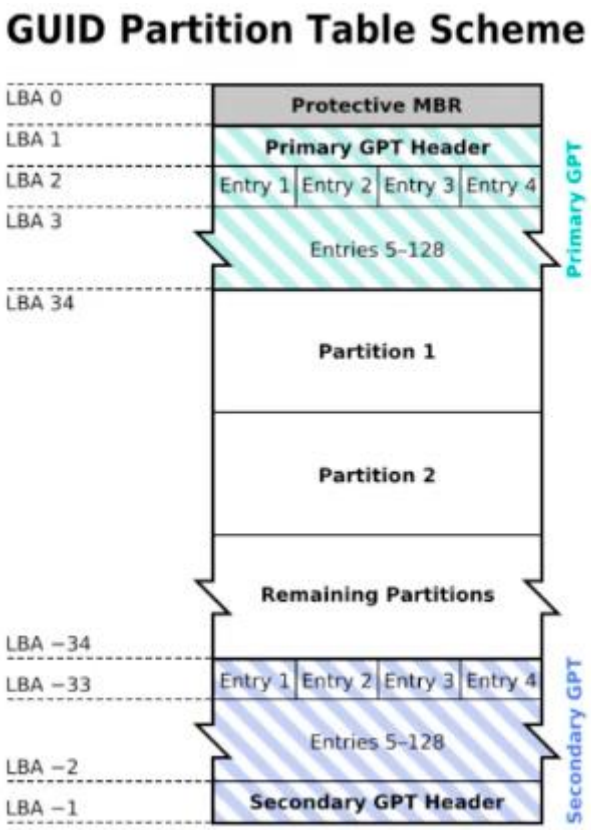
Один из родственных стандартов, применяющихся в Linux — UUID. Также 128-битный. Это universally unique identifier — универсальный уникальный идентификатор.

GUID — основа GPT.

GPT уже не содержит загружаемый код: эти функции отнесены к UEFI. UEFI — **Extensible Firmware Interface** (**EFI**) — новая прошивка ПЗУ, пришедшая на смену BIOS. Кроме того, GPT является частью стандарта UEFI. Тем не менее блок MBR присутствует в начале для совместимости и защиты от повреждения утилитами, не умеющими работать с GPT, но понимающими MBR. В совместимом MBR указан один раздел, охватывающий весь диск.

GPT не накладывает ограничений на разделы, поэтому такие понятия, как расширенные и первичные разделы, в работе с ним не используются.

Fdisk не работает с GPT, вместо него следует использовать **gdisk**.



Позволяет создавать разделы объемом до 9,4 Зб (зеттабайт). Приставка «тера-» означает 10^{12}, «зетта-» — 10^{21}.

Зачем нужна таблица разделов

- Для логического разделения дисков (C:, D:, E: в Windows, /dev/sda1, /dev/sd2, /dev/sda3 в Linux).
- Для загрузки диска — чтобы загрузить операционную систему, необходимо иметь таблицу разделов, чтобы можно было найти загрузчик операционной системы.

Если диск нужно использовать только для хранения информации, на нем не обязательно создавать структуру таблицы разделов. Файловые системы позволяют работать непосредственно с устройствами дисков.

Форматирование

Форматирование — эти создание разметки диска, а также создание файловой системы.

Различается низкоуровневое и высокоуровневое форматирование.

Низкоуровневое форматирование

Создает разметку дорожек и секторов на диске. В настоящее время, как правило, эта часть выполняется в заводских условиях и пользователям недоступна.

Высокоуровневое форматирование

Высокоуровневое форматирование записывает на диск (в раздел) файловую систему. После этого диск доступен для использования в ОС, поддерживающих файловую систему, в которой раздел был отформатирован.

Создание загрузочной записи может выполняться как при форматировании, так и отдельно специальными утилитами.

Восстановление информации

Так как высокоуровневое форматирование фактически изменяет только оглавление диска, потерянные данные в результате ошибочного форматирования теоретически можно восстановить. При этом возможны частичные потери иерархии, структуры и данных. Информацию позволяют восстановить такие программы, как **GetDataBack** для Windows (в данном случае для NTFS). Если вышел из строя контроллер диска, восстановление заключается в технической операции с использованием устройства-донора.

Удаление без возможности восстановления

С учетом особенностей системы для удаления конфиденциальных данных недостаточно удалить файл или отформатировать диск. Специальные технические средства позволяют восстановить данные даже после двукратной перезаписи секторов. Потому существуют утилиты, которые не просто удаляют файлы и разделы, но и принудительно перезаписывают нулями или случайной информацией трижды использованные ранее секторы.

Утилиты для работы с дисками

Разберем утилиты для работы с дисками в ОС GNU/Linux.: **fdisk**, **dd** и **hexdump**.

Некоторые из них присутствуют и в других операционных системах. **Dd** есть во всех UNIX-подобных системах, в том числе в MAC OS. **fdisk** есть еще и в DOS, и в Windows.

Утилита fdisk

Утилита присутствует во всех операционных системах. Появилась в UNIX, а при создании ОС DOS под влиянием UNIX была добавлена и туда. Работает в текстовом режиме и только с MBR — для работы с GPT необходим **gdisk**. **Fdisk** может повредить данные на вашем компьютере, поэтому использовать ее следует с осторожностью. Если быть точнее, сами данные не повредятся. Fdisk работает только с таблицей разделов, поэтому, если после ошибочных действий удастся восстановить эту таблицу, доступ к данным будет возвращен. Тем не менее даже зная, что **fdisk** не трогает сами данные, а только таблицу, используйте эту программу с большой осторожностью. Если правильную разметку не восстановить, то дальнейшая работа с файловой системой на этом диске будет невозможна.

```
root@ubuntu16:~# fdisk /dev/sda

Добро пожаловать в fdisk (util-linux 2.27.1).
Изменения остаются только в оперативной памяти, пока вы не решите их сохранить.
Будьте осторожны с использованием команды сохранения!


Команда (m для справки): p
Диск /dev/sda: 10 GiB, 10737418240 байтов, 20971520 секторов
Единицы измерения: секторов из 1 * 512 = 512 байтов
Размер сектора (логический/физический): 512 байт / 512 байт
I/O size (minimum/optimal): 512 bytes / 512 bytes
Тип метки диска: dos
Идентификатор диска: 0xeea25fd3

Устр-во    Загрузочный   Start Конец Секторы Size Id Тип
/dev/sda1                 2048 4196351 4194304   2G 83 Linux
/dev/sda2              4196352 6293503 2097152   1G 83 Linux
/dev/sda3              6293504 8390655 2097152   1G 83 Linux

Команда (m для справки):
```

Утилита dd

Одна из полезных утилит низкоуровневой работы с дисками в ОС GNU/Linux — **dd** — **dataset definition**.

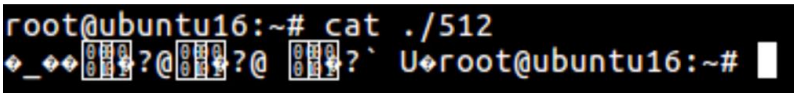
Работает непосредственно с дисковым устройством. Служит для чтения и записи информации. Альтернативное название: **dd** — **disk destroyer** — разрушитель диска. Если неосторожно использовать эту утилиту, можно распрощаться с данными навсегда. Записав данные не в то место на диске, можно уничтожить разметку, файловую систему или файлы. **Dd** не запрашивает подтверждение. Если же используем эту утилиту для чтения, ничего катастрофического не произойдет.

В UNIX-подобных операционных системах действует принцип: всё есть файл. С дисками тоже можно работать как со специализированными файлами, что данная утилита и делает. **dd** работает с файлом с начала диска или с указанного блока (файлы устройств в Linux могут быть символьными, которые считываются посимвольно, или блочными (поблочно). Файлы дисковых устройств — блочные.

Утилита hexdump

Утилита **dd** работает с сырыми данными, которые не отображаются в удобном для чтения формате. Для отображения данных используется другая утилита — **hexdump**. Она нужна потому, что терминал воспринимает двоичные данные как символы алфавита (а также символы псевдографики и управляющие конструкции ASCII: переводы строк, разные виды табуляций и т. д.). И если попытаться отобразить сырые двоичные данные на экран, выведется нечитаемый набор символов.

Пример вывода на экран двоичного (бинарного) файла:



Тот же самый файл при прочтении с помощью **hexdump**:

```
root@ubuntu16:~# hexdump -C ./512
00000000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001b0  00 00 00 00 00 00 00 00  d3 5f a2 ee 00 00 00 00  |........._......|
000001c0  01 01 83 3f 20 00 00 08  00 00 00 00 40 00 00 00  |...? .......@...|
000001d0  01 01 83 3f 20 00 00 08  40 00 00 00 20 00 00 00  |...? ...@... ...|
000001e0  01 01 83 3f 20 00 00 08  60 00 00 00 20 00 00 00  |...? ...`... ...|
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00000200
root@ubuntu16:~#
```

Видим смещения кодов, представления в шестнадцатеричной системе и в виде алфавитно-цифрового символа (если это возможно — бывает полезно, когда сырые данные содержат текстовую информацию, например имена файлов). Управляющие символы заменяются на точки .

Работа с утилитами fdisk, dd

Научимся работать с утилитой **fdisk**. При запуске указываем название диска, с которым будем работать:

```
# fdisk /dev/sdb
```

Или как в примере с видео:

```
# fdisk /dev/vda
```

```
root@ubuntu16:~# fdisk /dev/vda

Добро пожаловать в fdisk (util-linux 2.27.1).
Изменения остаются только в оперативной памяти, пока вы не решите их сохранить.
Будьте осторожны с использованием команды сохранения!


Команда (m для справки):
```

Появляется приглашение для ввода команды.

Команда **p** (достаточно нажать кнопку **p** и **Enter**) позволяет посмотреть разделы на указанном блочном устройстве.

```
root@ubuntu16:~# fdisk /dev/vda

Добро пожаловать в fdisk (util-linux 2.27.1).
Изменения остаются только в оперативной памяти, пока вы не решите их сохранить.
Будьте осторожны с использованием команды сохранения!


Команда (m для справки): p
Диск /dev/vda: 10 GiB, 10737418240 байтов, 20971520 секторов
Единицы измерения: секторов из 1 * 512 = 512 байтов
Размер сектора (логический/физический): 512 байт / 512 байт
I/O size (minimum/optimal): 512 bytes / 512 bytes
Тип метки диска: dos
Идентификатор диска: 0x9703d6c1

Устр-во    Загрузочный Start Конец Секторы Size Id Тип
/dev/vda1  *           2048 20969471 20967424  10G 8e Linux LVM

Команда (m для справки):
```

Мы видим всего один раздел диска **/dev/vda**, который называется **/dev/vda1** (если бы был еще один раздел, назывался бы **/dev/vda2** и т. д.). Он загрузочный.

Информация, которая отображается и которую мы можем оценить:

- в поле **Start** указано, что раздел начинается с 2048 сектора;
- видим в поле «Конец», что на секторе 20969471 раздел заканчивается;
- в поле «Секторы» видим их количество — 20967424;
- в поле Size видим объем — 10 G;
- идентификатор в шестнадцатеричной записи в поле Id — 8e;
- тип раздела: Linux LVM.

Чтобы понять, откуда эти данные берутся, прочитаем сырые данные с помощью **dd**.

Но сначала нужно выйти из **fdisk**. В этом поможет команда **q** (жмем **q** и **Enter**).

```
root@ubuntu16:~# fdisk /dev/vda

Добро пожаловать в fdisk (util-linux 2.27.1).
Изменения остаются только в оперативной памяти, пока вы не решите их сохранить.
Будьте осторожны с использованием команды сохранения!


Команда (m для справки): p
Диск /dev/vda: 10 GiB, 10737418240 байтов, 20971520 секторов
Единицы измерения: секторов из 1 * 512 = 512 байтов
Размер сектора (логический/физический): 512 байт / 512 байт
I/O size (minimum/optimal): 512 bytes / 512 bytes
Тип метки диска: dos
Идентификатор диска: 0x9703d6c1

Устр-во    Загрузочный Start Конец Секторы Size Id Тип
/dev/vda1  *           2048 20969471 20967424  10G 8e Linux LVM

Команда (m для справки): q
```

Теперь будем использовать конвейер из программы **dd**, которая прочитает данные с диска, и **hexdump**, которая позволит их отобразить в читаемом виде.

Команда, которую мы будем использовать:

```
# dd if=/dev/vda bs=1 count=512 |hexdump -C
```

Вертикальная черта между командой **dd** и командой **hexdump** означает, что обе программы будут запущены, но результат вывода первой (**dd**) будет поступать не на экран, а на вход второй программы (**hexdump**). Она обработает сырые данные и уже сама будет отображать их в удобочитаемом виде для нас на экране.

Параметры **dd**, которые мы укажем:

- **if=/dev/vda** — путь для входного файла. Здесь мы читаем не раздел диска, а именно физический диск;
- **bs=1** — раздел блока — 1 байт, для удобства;
- **count=512** — общее количество блоков, которые надо считать. Так как MBR занимает 512 байт и содержится в 1 секторе, мы считаем 512 блоков по 1 байту, фактически 512 байт.

Hexdump переведет двоичные данные в шестнадцатеричный формат:

```
00000040  4c 02 cd 13 ea 00 7c 00  00 eb fe 00 00 00 00 00  |L.....|.........|
00000050  00 00 00 00 00 00 00 00  00 00 00 80 01 00 00 00  |................|
00000060  00 00 00 00 ff fa 90 90  f6 c2 80 74 05 f6 c2 70  |...........t...p|
00000070  74 02 b2 80 ea 79 7c 00  00 31 c0 8e d8 8e d0 bc  |t....y|..1......|
00000080  00 20 fb a0 64 7c 3c ff  74 02 88 c2 52 bb 17 04  |. ..d|<.t...R...|
00000090  f6 07 03 74 06 be 88 7d  e8 17 01 be 05 7c b4 41  |...t...}.....|.A|
000000a0  bb aa 55 cd 13 5a 52 72  3d 81 fb 55 aa 75 37 83  |..U..ZRr=..U.u7.|
000000b0  e1 01 74 32 31 c0 89 44  04 40 88 44 ff 89 44 02  |..t21..D.@.D..D.|
000000c0  c7 04 10 00 66 8b 1e 5c  7c 66 89 5c 08 66 8b 1e  |....f..\|f.\.f..|
000000d0  60 7c 66 89 5c 0c c7 44  06 00 70 b4 42 cd 13 72  |`|f.\..D..p.B..r|
000000e0  05 bb 00 70 eb 76 b4 08  cd 13 73 0d 5a 84 d2 0f  |...p.v....s.Z...|
000000f0  83 d0 00 be 93 7d e9 82  00 66 0f b6 c6 88 64 ff  |.....}...f....d.|
00000100  40 66 89 44 04 0f b6 d1  c1 e2 02 88 e8 88 f4 40  |@f.D...........@|
00000110  89 44 08 0f b6 c2 c0 e8  02 66 89 04 66 a1 60 7c  |.D.......f..f.`||
00000120  66 09 c0 75 4e 66 a1 5c  7c 66 31 d2 66 f7 34 88  |f..uNf.\|f1.f.4.|
00000130  d1 31 d2 66 f7 74 04 3b  44 08 7d 37 fe c1 88 c5  |.1.f.t.;D.}7....|
00000140  30 c0 c1 e8 02 08 c1 88  d0 5a 88 c6 bb 00 70 8e  |0........Z....p.|
00000150  c3 31 db b8 01 02 cd 13  72 1e 8c c3 60 1e b9 00  |.1......r...`...|
00000160  01 8e db 31 f6 bf 00 80  8e c6 fc f3 a5 1f 61 ff  |...1..........a.|
00000170  26 5a 7c be 8e 7d eb 03  be 9d 7d e8 34 00 be a2  |&Z|..}....}.4...|
00000180  7d e8 2e 00 cd 18 eb fe  47 52 55 42 20 00 47 65  |}.......GRUB .Ge|
00000190  6f 6d 00 48 61 72 64 20  44 69 73 6b 00 52 65 61  |om.Hard Disk.Rea|
000001a0  64 00 20 45 72 72 6f 72  0d 0a 00 bb 01 00 b4 0e  |d. Error........|
000001b0  cd 10 ac 3c 00 75 f4 c3  c1 d6 03 97 00 00 80 20  |...<.u......... |
000001c0  21 00 8e fe ff ff 00 08  00 00 00 f0 3f 01 00 00  |!...........?...|
000001d0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00000200
512+0 записей получено
512+0 записей отправлено
512 байт скопировано, 0,00483908 s, 106 kB/s
root@ubuntu16:~#
```

Видим, что последние два байта в шестнадцатеричном виде — 55 aa. То есть это MBR, и она корректна.

Самая интересная область — таблица разделов MBR. Именно на ее основе **fdisk** и предоставляет нам информацию.

С помощью **dd** можно извлечь любую информацию. Нас интересует таблица. Так как есть только один раздел, это будет первая таблица.

```
# dd if=/dev/vda skip=446 bs=1 count=512 |hexdump -C
```

- **if=/dev/vda** — извлекаем данные с того же физического диска **/dev/vda**.
- **skip=446 байт** — пропускаем 446 байт загрузчика, после которого содержится запись первого раздела.
- **bs=1** — по-прежнему будем считывать блоки размером в 1 байт.
- **count=16**.

```
root@ubuntu16:~# dd if=/dev/vda skip=446 bs=1 count=16 | hexdump -C
16+0 записей получено
16+0 записей отправлено
00000000  80 20 21 00 8e fe ff ff  00 08 00 00 00 f0 3f 01  |. !.........?.|
00000010
16 байт скопировано, 0,000456463 s, 35,1 kB/s
root@ubuntu16:~#
```

Разберем значения байт:

- 80 — признак того, что раздел загрузочный. Если 0 — то не загрузочный;
- 20 21 00 — номер сектора, с которого начинается раздел в системе CHS;
- 8e — код типа раздела, соответствует типу Linux;
- **fe ff ff** — сектор, в котором раздел заканчивается в системе адресации CHS;
- **00 08 00 00** — начало в адресации LBA. В архитектуре Intel порядок байт Little Endian. Поэтому адрес в понятном человеку виде (но по-прежнему в шестнадцатеричной системе) 8 00. Переведем в десятичную: 2048;
- **00 f0 3f 01** — информация об общем количестве секторов в разделе. Так же нужно отобразить в Little Endian **01 3f f0 00** и перевести в десятичную запись. Так мы можем понять ограничение на размер. Соответственно, 4 байта, выделенные на хранение количества секторов, это 32 бита. Максимальное значение, которое могут хранить 4 байта, это 2^{32}. 1 сектор — 512 байт), так что максимальный размер раздела — $2^{32} \times 512 = 2$ TiB. Поэтому в MBR не может быть раздела более 2 Тб.

Практическое задание

Ответьте на вопросы в Google Docs, максимально подробно объясняя, почему вы так считаете. Приложите ссылку на документ к уроку.

1. Что такое MBR?
2. Можно ли работать с диском без MBR?
3. Почему в MBR нельзя создать раздел больше 2 TiB?

4. Влияет ли изменение MBR на изменение данных в файловой системе?
5. Возможно ли в MBR увеличить первый раздел из двух?
6. Откуда **fdisk** берет информацию о диске и разделах на нем?
7. Почему для работы с бинарными данными нужна специальная программа (например, **hexdump**)?
8. Почему, работая с утилитой **dd**, нужно быть очень внимательным?
9. Почему при работе с MBR следует обращаться к физическому, а не к логическому диску?
10. Что такое GPT?

Задачи со * предназначены для продвинутых учеников.

Дополнительные материалы

1. Главная загрузочная запись — Master Boot Record (MBR).
2. GPT vs MBR.
3. Man pages.
4. Команда dd и все, что с ней связано.
5. Интерактивная система просмотра системных руководств (man-ов).
6. Команда hexdump.

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. GUID.
2. UUID.
3. Пишем свой загрузочный сектор.
4. Главная загрузочная запись.
5. GPT.
6. Главная Загрузочная Запись - Master Boot Record (MBR).
7. GPT vs MBR.